



**Lab Continuous Assessment Task -II - Prompt Engineering & LLM Behavior
Analysis**

“AI Concept Tutor”

by

Roll No:16010123324 Shreya Menon
Roll No: 16010123325 Shreyans Tatiya

For the subject

Artificial Intelligence

Department of Computer Engineering
Academic Year 2025-26

AI Concept Tutor

Task domain and Problem statement chosen:

Task Domain

The task domain is **Intelligent Educational Tutoring**, a subfield of Artificial Intelligence focused on delivering personalized concept explanations and adaptive learning experiences.

Problem Statement

The problem is to design an AI Concept Tutor that can explain technical and academic concepts at multiple difficulty levels while ensuring outputs are clear, structured, and safe.

The system should:

- Adapt explanations based on the learner's understanding level
- Present information in a well-structured and step-by-step manner
- Maintain clarity to avoid confusion in complex topics
- Ensure safety by preventing misleading, harmful, or incorrect explanations

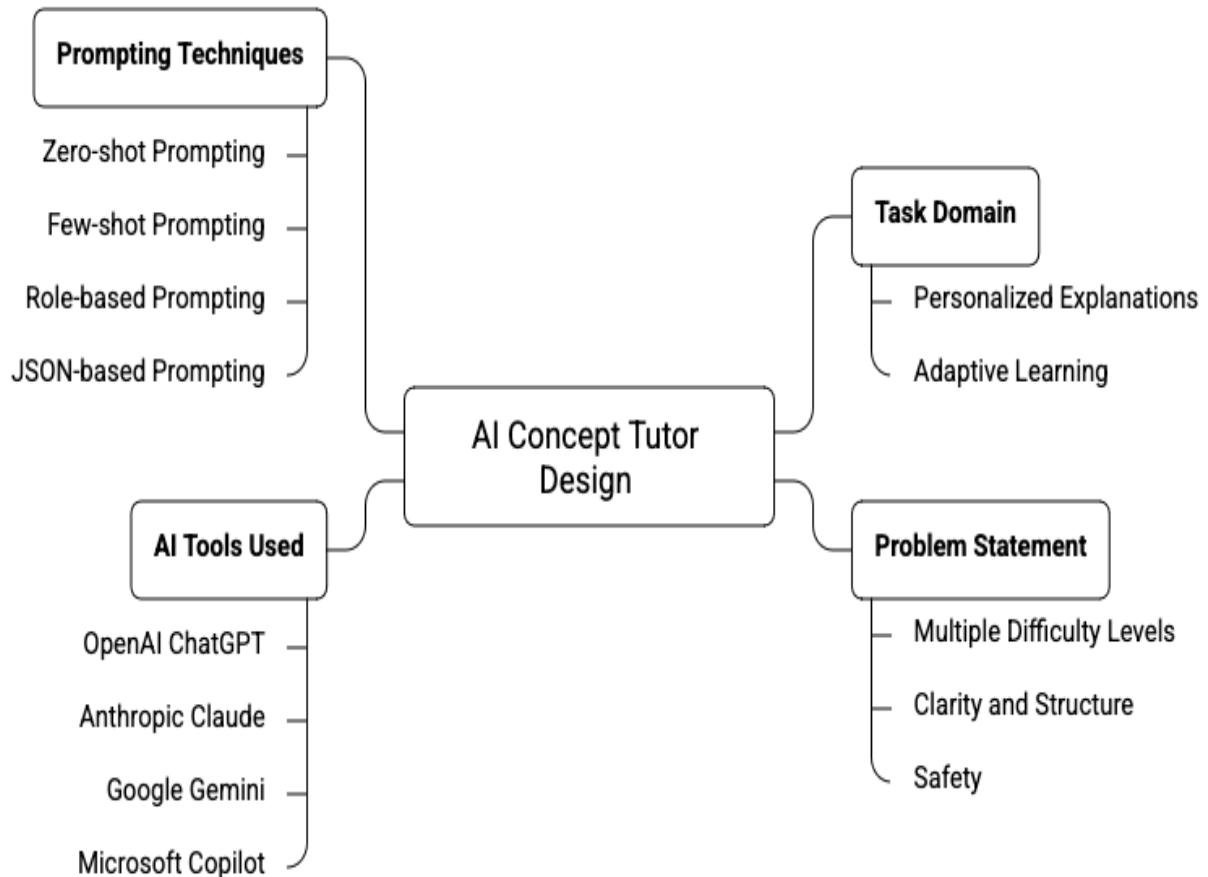
List of AI Tools used during prompt engineering exploration:

- OpenAI ChatGPT
- Anthropic Claude
- Google Gemini
- Microsoft Copilot

List of Prompting techniques explored:

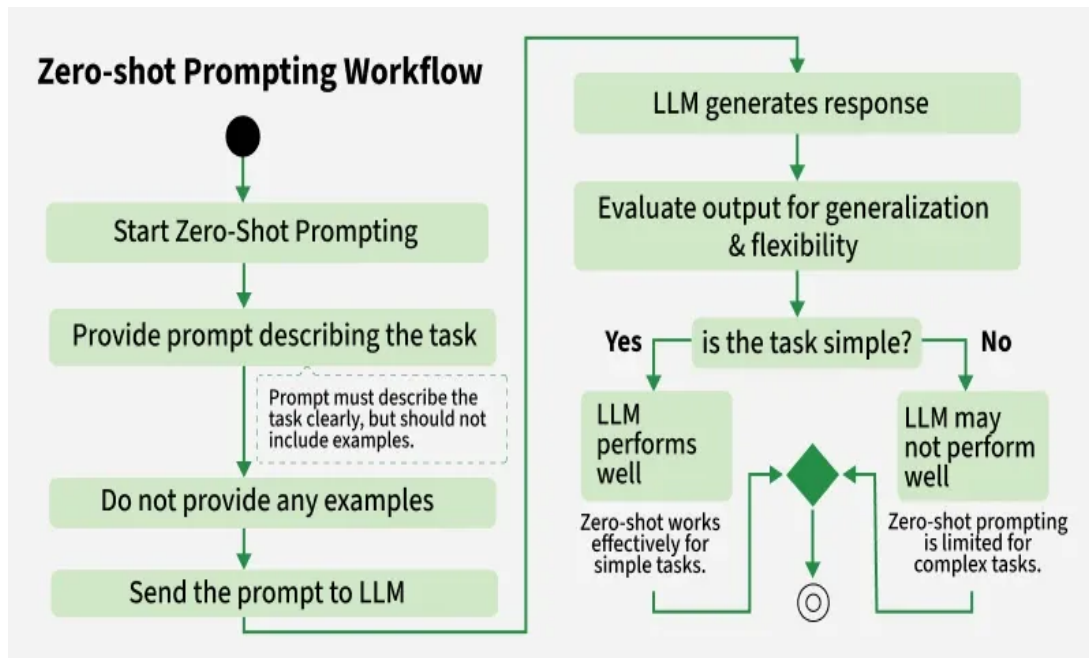
- Zero-shot prompting
- Few-shot prompting
- Role-based prompting
- JSON-based prompting

AI Concept Tutor Design and Exploration



1. Zero-shot Prompt

Define Zero-shot Prompt: A zero-shot prompt gives the model only the task instruction and constraints, without providing solved examples in advance.



Prompt 1 :

Focus: Easy input

Prompt statement:

You are a supportive academic tutor.

Explain AI to a beginner in simple terms.

Role: AI tutor

Output format: bullet points + example

Constraints: max 150 words

Tone: Simple and friendly

Safety: Avoid confusing jargon

Output from ChatGPT:

- Artificial Intelligence (AI) means **machines doing tasks that usually need human thinking**
- It helps computers **learn from data and make decisions**
- AI doesn't "think" like humans—it follows patterns and rules from what it learns

- The more data it gets, the **better it improves over time**
- AI is used in things like **voice assistants, recommendations, and self-driving cars**

Example: Movie Recommendations

- When you watch movies on a platform, AI notices what you like
- It compares your choices with others who have similar tastes
- Then it suggests new movies you might enjoy

So, AI is like a **smart helper** that learns from experience and gives useful suggestions

Output from Claude:

What is AI?

AI (Artificial Intelligence) = teaching computers to think and learn like humans!

- **It learns from examples** — just like you learned that fire is hot by experience
- **It recognizes patterns** — spots a cat in a photo after seeing millions of cat pictures
- **It makes decisions** — like Netflix suggesting a show you'll probably love
- **It understands language** — that's how I'm chatting with you right now!
- **It gets better over time** — the more data it sees, the smarter it becomes

Simple example:

You show a child 100 pictures of dogs and cats. Soon they can tell the difference. AI works the **same way** just with computers instead of kids!

Bottom line: AI is a computer that learns to do smart things it's not magic, just clever math!

Output from Gemini:

Hello! Let's explore Artificial Intelligence (AI) together. You'll grasp this in no time!

- **What it is:** AI helps computers do tasks that usually require human brainpower.
- **How it learns:** Instead of following a strict rulebook, it looks at huge amounts of information to find patterns.
- **What it does:** It uses those patterns to make decisions, answer questions, or recognize objects.

Example: Imagine teaching a child what a dog is. You don't give them a checklist of dog traits; you just point out dogs! Over time, they naturally learn to spot one. AI works the exact same way. If we show a computer millions of dog photos, it learns to recognize them on its own.

Output from Co-pilot:

- **AI definition:** Artificial Intelligence (AI) is when computers are designed to “think” or “learn” in ways similar to humans.
- **Learning from data:** AI looks at lots of examples (like photos or sentences) and finds patterns.
- **Making decisions:** Once it learns, AI can make predictions or choices, such as suggesting a movie or recognizing a face.
- **Everyday uses:** AI powers voice assistants, spam filters in email, and even self-driving cars.
- **Key idea:** AI doesn’t “think” like us—it follows rules and patterns we teach it, but it can process huge amounts of information very quickly.

Example: Imagine teaching a child to recognize cats. You show them many cat pictures, and they learn what makes a cat. AI works the same way—feed it thousands of cat photos, and it learns to spot cats on its own.

Model	Analysis
ChatGPT	Most structured and aligned with prompt
Claude	Engaging but slightly informal
Gemini	Engaging but slightly informal
Copilot	Balanced but slightly repetitive

Prompt 2 :

Focus: Hard input

Prompt statement:

You are a supportive academic tutor.

Explain the concept of overfitting in machine learning to:

- *A beginner*
- *An intermediate learner*

- *An advanced learner*

Role: AI tutor

Output format: Separate sections for each level + example

Constraints: max 180 words

Tone: simple and clear (adjust per level)

Safety: avoid misleading or incorrect explanations

Output from ChatGPT:

Beginner

- Overfitting happens when a model **memorizes training data** instead of learning patterns
- It does very well on seen data but **fails on new data**
- Like studying answers instead of understanding the topic

Intermediate

- The model captures **noise and small details** in training data
- This leads to **low training error but high test error**
- Common causes: too complex model, small dataset
- Fixes: more data, regularization, simpler models

Advanced

- Overfitting = **high variance problem**
- Model fits $f(x) + \text{noise}(x) + \text{noise}(x) + \text{noise}$, not just true function
- Leads to poor **generalization**
- Controlled using techniques like **L1/L2 regularization, dropout, cross-validation**

Example

- Predict exam scores based on study hours
- Overfit model memorizes exact past scores (even random variations)
- When a new student comes, prediction is inaccurate because it learned noise, not the real trend

Output from Claude:

Overfitting in Machine Learning

Beginner — *Keep it simple!*

Overfitting is when a model **memorizes** instead of **learns**.

- Like a student who memorizes exam answers word-for-word
- They fail when questions are slightly rephrased!

Fix: Give the model more varied data to truly *learn* from.

Intermediate — *Getting technical!*

Overfitting happens when a model learns the **training data too well** — including its noise — and performs **poorly on new data**.

- High training accuracy , low test accuracy
- The model captures noise, not real patterns

Fix: Use techniques like **cross-validation**, **dropout**, or **regularization**.

Advanced — *Deep dive!*

Overfitting reflects a **high-variance, low-bias** model that fails to generalize.

- The hypothesis function fits training distribution too tightly
- Regularization (L1/L2) penalizes model complexity
- Bayesian approaches help by placing **priors** on parameters

Fix: Tune the **bias-variance tradeoff** using regularization + ensemble methods.

Output from Gemini:

Let's break down overfitting!

Beginner - Overfitting is when a computer memorizes data instead of actually understanding it. It looks perfect in practice but fails in reality.

- **Example:** Memorizing the exact questions on a practice math test, but failing the real exam because the numbers changed slightly.

Intermediate - Overfitting happens when a machine learning model learns the random "noise" in training data rather than the true underlying pattern. It has low bias but high variance, which causes poor generalization to new datasets.

- **Example:** Drawing a complex, squiggly line that perfectly connects every single dot on a scatter plot (including the outliers), instead of drawing a smooth, general trend line.

Advanced - Overfitting is a generalization failure where a model's capacity (e.g., high VC dimension) exceeds the dataset's true signal. The model fits the irreducible error, minimizing empirical risk on the training set while expected risk (out-of-sample test error) spikes.

- **Example:** Fitting a 20th-degree polynomial to sparse, noisy data. The training error drops to exactly zero, but the function oscillates wildly between points, yielding massive errors on unseen data.

Output from Co-pilot:

For a Beginner

- **Overfitting** happens when a machine learning model memorizes training data instead of learning general rules.
- It works well on the data it has seen but fails on new data.
- Think of it like memorizing a poem without understanding the meaning.

Example: A student memorizes practice questions but struggles when the exam has different ones.

For an Intermediate Learner

- **Overfitting** occurs when a model has high variance: it adapts too closely to training data patterns.
- Causes include overly complex models, small datasets, or lack of regularization.
- The result: low training error but high test error.

Example: A deep decision tree perfectly classifies training data but misclassifies new samples.

For an Advanced Learner

- **Overfitting** reflects poor generalization due to excessive model complexity relative to data distribution.
- It can be analyzed in terms of bias-variance tradeoff and generalization bounds.
- Mitigation strategies include cross-validation, dropout, L1/L2 regularization, and Bayesian approaches.

Example: A neural network with too many parameters fits training noise, reducing predictive reliability on unseen data.

Model	Analysis
ChatGPT	Clear structure and good separation of levels. However, beginner and intermediate explanations are slightly similar, reducing distinction.
Claude	Best differentiation across levels with smooth progression. Slightly informal tone but strong conceptual clarity.

Gemini	Strong advanced explanation with depth. However, uses heavy terms (e.g., VC dimension), making it less aligned with clarity constraint.
Copilot	Balanced and consistent explanations across levels. Slightly repetitive and less variation in depth between sections.

Prompt Failures & Biases Identified:

- **Inconsistent level differentiation** – some outputs use similar depth across levels
- **Over-simplification** at beginner level
- **Over-complexity** at advanced level (too much jargon)
- **Structure issues** – not all clearly separate levels or use same example

Prompt Refinement:

Refined Prompt:

Explain overfitting at three levels:

- *Beginner (simple, no jargon)*
 - *Intermediate (basic technical terms)*
 - *Advanced (include bias-variance)*
- Use one example for all. Keep sections clearly separate.*

Before vs After Comparison

The original prompt gives correct explanations but lacks clear distinction between levels and consistency in structure. The refined prompt improves clarity by enforcing different depths, structured sections, and a common example, making the output more suitable for adaptive learning.

Prompt 3 :

Focus: Ambiguous input

Prompt statement:

You are a supportive academic tutor.

Output format: short explanatory paragraph. Constraints: 80-110 words, simple language. Tone: neutral and helpful

Safety: if the request is ambiguous, say so before answering. Teach me about power

Output from ChatGPT:

“Power” can mean different things, so I’ll assume you mean it in a general science context. Power is the **rate at which work is done or energy is used**. In simple terms, it tells you how fast something uses or transfers energy. For example, a higher-watt light bulb uses energy faster and shines brighter. The unit of power is the **watt (W)**. In electricity, power = voltage × current. In everyday life, power helps compare how quickly machines, devices, or even people perform tasks.

Output from Claude:

"Power" is a broad topic — it appears in several different fields, so I want to make sure I teach you the most relevant one! Here are the most common meanings:

- **Physics** — Power as energy transferred over time (watts, electricity)
- **Politics/Social Science** — Power as influence or authority over others
- **Mathematics** — Power as exponents (e.g., $2^3 = 8$)

Could you clarify which one you mean? Once you let me know, I'll give you a clear, focused explanation tailored just for you!

Output from Gemini:

Power is a concept that appears in many areas, so I want to make sure I’m giving you the right information. Since "power" can mean different things, I will focus on the physics definition.

In physics, power is how quickly work is done or energy is used. Imagine two people carrying the same heavy box up a flight of stairs. They both do the same amount of work because the box and the stairs are the same. However, if one person runs up the stairs and the other walks, the runner uses more power. This is because power measures the speed at which the energy is used. We use a unit called the "watt" to measure power.

Output from Co-pilot:

Power, in simple terms, is the ability to influence or control what others do. It can be used positively, like guiding a team, or negatively, like forcing someone against their will.

Power is a central idea in politics and society. At its core, it means having the capacity to shape decisions, behaviors, or outcomes. Unlike authority, which is recognized as legitimate, power can be exercised with or without approval. Thinkers like Max Weber described power as the ability to carry out one's will even when others resist. In everyday life, power shows up in many forms: a teacher guiding students, a government making laws, or even a friend persuading you to try something new. The key point is that power is about influence—sometimes through persuasion, sometimes through control. Understanding it helps explain how relationships, communities, and nations function.

Model	Analysis
ChatGPT	Correctly identifies ambiguity and proceeds with a reasonable assumption (physics). Follows instructions well but does not explicitly ask for clarification.
Claude	Best handling of ambiguity—clearly lists possible meanings and asks for clarification. However, does not provide the final explanation, partially missing task completion.
Gemini	Acknowledges ambiguity and selects physics meaning. Provides a clear and detailed explanation, but assumes intent instead of confirming.
Copilot	Directly assumes a social/political meaning without addressing ambiguity. Violates safety instruction by not acknowledging multiple interpretations.

Prompt Failures & Biases Identified:

- **Ambiguity (Primary Issue)**
The term “*power*” is unclear (physics, politics, math), leading to different interpretations across models.
- **Inconsistent Handling of Ambiguity**
Some models (Claude) ask for clarification, while others (ChatGPT, Gemini, Copilot) assume a meaning.
- **Incorrect Assumption**
Models that directly choose one meaning (physics or social) may not match user intent.

Prompt Refinement:

Refined prompt:

*Teach me about **power in physics**.
If multiple meanings exist, briefly mention them but focus on physics.
Explain in a short paragraph (80–110 words) using simple language.*

Before vs After Comparison

The original prompt leads to varied interpretations due to ambiguity, resulting in inconsistent outputs across models. Some assume meanings while others request clarification, reducing uniformity. The refined prompt removes ambiguity by specifying the domain, leading to more consistent, relevant, and complete responses while still maintaining simplicity.

Prompt 4 :

Focus: Contradictory input

Prompt statement:

You are a supportive academic tutor.

Explain recursion in detail using technical terms.

Output format: very simple explanation for a beginner

Constraints: avoid all technical jargon

Tone: simple and friendly

Safety: ensure clarity

Output from ChatGPT:

Recursion is when something solves a problem by using the same steps again and again on smaller parts of the problem.

Think of it like this:

You have a big task. Instead of doing it all at once, you break it into smaller similar tasks, and keep doing that until the task becomes very simple.

There is always:

A smallest case where you stop

A way to move toward that smallest case

For example, imagine counting down:

To handle 3, you first handle 2, then 1, then stop.

So, recursion is just a way of solving problems by repeating the same idea on smaller pieces until it's easy to finish.

Output from Claude:

What is Recursion?

Think of recursion like **Russian nesting dolls!**

Imagine this:

You open a doll — there's a smaller doll inside. You open that one — another smaller doll inside. This keeps going **until you reach the tiniest doll** that can't be opened!

In programming, it's the same idea:

- A **function** (a small task the computer does) **calls itself** to solve a smaller piece of the same problem
- It keeps doing this **until it hits a simple answer** it already knows

Everyday Example:

Imagine counting stairs going **down**:

- "How many stairs are left?" → *One less than before!*
- Keep asking the same question until you reach **stair zero!**

Two simple rules of recursion:

- It **repeats itself** on a smaller version of the problem
- It always has a **stopping point** — otherwise it goes forever!

Output from Gemini:

At its heart, **recursion** is simply a way of solving a large puzzle by breaking it down into smaller and smaller versions of the *exact same puzzle*, until the pieces are so tiny that the answer is completely obvious.

The "Standing in Line" Example

Imagine you want to know what your exact position is in a very long line, but you aren't allowed to step out of line to count. You can only talk to the person directly in front of you.

Here is how you would use recursion to find out:

1. You ask the person in front of you, "What number are you?"
2. They don't know either, so they ask the person in front of them the exact same question.
3. This repeating chain continues all the way to the very front of the line.
4. The first person in line knows they are at the front, so they turn around and say, "I am number one!"
5. The second person hears this, adds one, and tells the person behind them, "I am number two!"
6. This ripples all the way back until the person right in front of you tells you their number, and you finally know yours.

Instead of trying to figure out the whole giant problem at once, recursion just repeats one simple, tiny action over and over until the job is completely done. You just have to make sure there is a clear stopping point (like the first person in line) so the questions don't go on forever!

Output from Co-pilot:

Recursion is simply a way of solving a problem by breaking it into smaller versions of itself. Instead of tackling everything at once, you let the problem “call itself” again and again until it becomes so small that the answer is obvious. Then, those small answers are combined to solve the bigger one.

Example: Imagine stacking boxes. To find what’s inside the bottom box, you must first open the top one, then the next, and so on. Each step is the same action—open a box—until you reach the last one. That’s recursion: repeating the same process until you reach a simple end.

Model	Analysis
ChatGPT	Follows beginner-friendly constraint well and keeps explanation simple. Ignores “technical terms” due to contradiction, leading to loss of depth.
Claude	Uses strong analogy (nesting dolls) and clear structure. Slightly better balance, but still prioritizes simplicity over technical detail.
Gemini	Most detailed explanation with step-by-step example. However, slightly exceeds simplicity constraint due to length and elaboration.
Copilot	Concise and balanced explanation. Clear but less detailed and less illustrative compared to others.

Prompt Failures & Biases Identified:

- Contradictory Instructions – asks for “technical terms” but also “avoid jargon”
- Priority Ambiguity – unclear which constraint to follow
- Inconsistent Model Behavior – all models ignored “technical terms” and chose simplicity

Prompt Refinement:

- **Refined prompt:**

Explain recursion to a beginner using simple language.

You may include one or two basic technical terms, but explain them clearly.

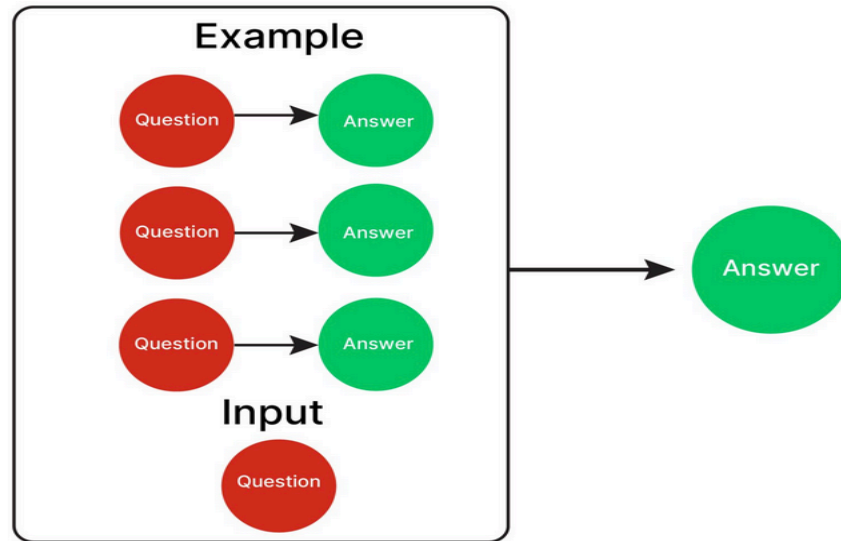
Use a short example.

- **Before vs after output comparison:**

The original prompt creates confusion due to conflicting instructions, leading models to ignore technical depth and prioritize simplicity. This results in consistent but incomplete adherence to the prompt. The refined prompt resolves the contradiction by clearly prioritizing beginner-friendly explanation while allowing limited technical terms, leading to more balanced, clear, and controlled outputs.

2. Few-shot Prompt

Define Few-shot prompt: A Few-shot prompt provides a few input-output examples before the actual task to guide the model's response style and structure.



Prompt 1 :

Focus: Easy Input

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Examples:

Input: Explain stack simply → Output: LIFO, like a stack of plates

Input: Explain queue simply → Output: FIFO, like a line

Task: Explain tree data structure

Safety: Avoid confusing jargon

Output from ChatGPT:

A tree is a way of organizing data like a **family tree**

- It starts with one main point called the **root**
- From it, smaller parts branch out, like **parent** → **children**
- Each point can connect to multiple others, forming levels

- There are no loops—you can only move **downward along branches**

Simple idea:

Like folders on your computer—one main folder, inside it more folders, and inside those, even more files and folders

Output from Claude:

Hierarchical, like a family tree — one root, branching into parents, children, and leaves!

Output from Gemini:

Output: Hierarchical, like a family tree or an organization chart

Output from Co-pilot:

A **tree data structure** is a way of organizing information that looks like an upside-down tree. It starts with a single point called the **root**, and from there branches spread out into smaller parts called **nodes**. Each node can connect to other nodes, forming a hierarchy.

Think of it like a **family tree**: the root is a grandparent, their children are the next level, and grandchildren form further branches. This structure makes it easy to search, sort, and organize data because you can move step by step down the branches.

Simple example:

- Root → “Fruit”
- Branches → “Citrus” and “Berries”
- Leaves → “Orange,” “Lemon,” “Strawberry,” “Blueberry”

Model	Analysis
ChatGPT	Best balance follows example style while adding slightly more detail. Clear, structured, and beginner-friendly.
Claude	Very concise and matches pattern closely, but too short lacks explanation depth.

Gemini	Strictly follows pattern (very short), but overly minimal almost no explanation.
Co-pilot	Most detailed explanation, but does not strictly follow example style (too long).

Prompt Failures & Biases Identified:

- **Pattern overfitting** – some models copy examples too rigidly (Claude, Gemini)
- **Inconsistent output length**

Prompt Refinement:

- **Refined prompt:**

Explain tree data structure in the same style as examples:

- *One-line definition*
- *Short explanation (2–3 points)*
- *One simple analogy*

- **Before vs after output comparison:**

The original prompt leads to inconsistent outputs, with some models being too brief and others too detailed. The refined prompt improves consistency by clearly defining structure and expected depth, resulting in more uniform and balanced responses.

Prompt 2 :

Focus: Difficult

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Examples:

Beginner: Simple explanation of recursion

Advanced: Technical explanation of recursion

Task: Explain dynamic programming at 2 levels

Output from ChatGPT:

Beginner

Dynamic programming means **saving answers to small problems** so you don't solve them again and again. It makes things faster by reusing work.

Advanced

Dynamic programming optimizes problems with **overlapping subproblems and optimal substructure**. Solutions are built using **memoization (top-down)** or **tabulation (bottom-up)** to reduce time complexity.

Example

Fibonacci numbers:

Instead of recalculating values repeatedly, store results like $F(1)$, $F(2)$, $F(3)$... and reuse them to compute the next values quickly.

Output from Claude:

Beginner

Remembering past work to avoid doing it again — like saving answers in a notebook!

Calculating $2+2$ once, writing it down, never solving it again!

Advanced

An optimization technique exploiting **optimal substructure** and **overlapping subproblems** via **top-down memoization** or **bottom-up tabulation**.

Bellman-Ford and knapsack problems solved in polynomial time by storing state transitions in a DP table!

Output from Gemini:

Beginner Dynamic programming is simply a method of solving a large problem by breaking it into smaller pieces, solving those pieces, and **writing the answers down** so you never have to solve them twice.

Imagine I ask you to calculate $1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$. You count them up and say "8." If I then append another $+ 1$ to the end of the equation, you don't recount all the ones from the beginning. You just take your remembered answer (8), add 1, and say "9." That is dynamic programming: using past answers to save future work.

Advanced Mathematically and algorithmically, dynamic programming is a technique that trades space complexity for time complexity. It optimizes recursive algorithms by reducing their time bounds—often from exponential time $O(2^n)$ down to polynomial time $O(nk)$.

An advanced view treats DP as traversing a Directed Acyclic Graph (DAG) of state dependencies. To implement it, you must define:

1. **The State:** A set of variables that uniquely identifies a subproblem.
2. **The Recurrence Relation (Bellman Equation):** The mathematical function defining how a complex state transitions into simpler, smaller states.
3. **The Base Case:** The trivial, atomic state where the recursion bottoms out.

DP can be implemented top-down using **memoization** (passing state through a recursive call stack and caching returns) or bottom-up using **tabulation** (iteratively filling an n-dimensional array starting from the base cases to avoid call stack overhead).

Output from Co-pilot:

Beginner: Dynamic programming is a way to solve problems by breaking them into smaller parts and remembering the answers to those parts. Instead of solving the same piece many times, you store the result and reuse it.

Example: If you keep forgetting a phone number, you write it down once and look at it whenever needed.

Advanced: Dynamic programming formalizes recursive problem-solving by using memoization or tabulation to reduce time complexity. It leverages the principle of optimality, ensuring that global solutions can be constructed from optimal local solutions.

Example: In shortest path algorithms like Bellman-Ford, dynamic programming ensures efficient computation by iteratively refining solutions using stored sub-results.

Model	Analysis
ChatGPT	Good level separation and clear explanation. Balanced but slightly brief compared to others.
Claude	Strong analogy and clear distinction. Advanced part is concise but slightly less detailed.
Gemini	Most detailed and technically rich. Clearly differentiates levels but slightly verbose.
Co-pilot	Well-balanced explanation with examples. Good clarity but less depth than Gemini.

Prompt Failures & Biases Identified:

- **Uneven depth across models**
 - **Verbosity variation** (Gemini too long, others shorter)
 - **Example inconsistency** (not all include or align examples)

Prompt Refinement:

- **Refined prompt:**

Explain dynamic programming at two levels:

- *Beginner (simple analogy, no jargon)*
 - *Advanced (include optimal substructure, overlapping subproblems)*
- Include one common example for both. Limit response length.*

- **Before vs after output comparison:**

The original prompt produces correct explanations but lacks control over depth and length, resulting in inconsistent outputs across models. The refined prompt improves consistency by clearly defining expected concepts, structure, and constraints, leading to more balanced and comparable responses.

Prompt 3 :

Focus: Ambiguous

Prompt statement:

Output format: short paragraph

Constraints: simple language, 80–110 words Tone: neutral and helpful

Safety: if the term is ambiguous, clarify before answering

Examples:

Example 1: Input: Explain “cell”

Output: A “cell” can mean different things. In biology, it is the basic unit of life...

Example 2: Input: Explain “network”

Output: The term “network” has multiple meanings. In computers, it refers to connected devices...

Task: Explain “bias” to a school student

Output from ChatGPT:

The word “bias” can mean different things depending on the context. In everyday life, it often means having an **unfair preference or opinion** about something or someone without good reason, like favoring one friend over others. In subjects like science or data, bias means a **systematic error** that affects results, making them less accurate. For example, if a survey only asks one type of group, the results may be biased. So, bias can either be about **unfair thinking** or **errors in information**, depending on how it is used.

Output from Claude:

"Bias" can mean different things depending on the subject, so here's a quick breakdown!

In **everyday life**, bias means favoring something unfairly — like always picking your best friend first for a team, even if others are better. In **statistics**, bias means your data is lopsided — like surveying only boys about a topic that affects everyone. In **AI/machine learning**, bias means the model makes unfair assumptions based on skewed training data.

All three share the same core idea — **an unfair lean in one direction** that distorts the truth!

Output from Gemini:

The word “bias” can mean a few different things. Most commonly, it means having an unfair preference for or against someone or something. For example, if a referee's favorite team is playing, their personal bias might make them judge the game unfairly. In math and science, bias happens when an experiment is set up in a way that gives an incorrect result, like only surveying your friends instead of the whole school. In technology, computer programs can also show bias if

they learn from unfair data. Overall, bias simply means leaning heavily toward one side instead of being completely neutral and fair.

Output from Co-pilot:

The word **bias** can mean different things, so let's clarify. In everyday life, bias is when someone favors one side or opinion more than another, often unfairly. For example, if a teacher always thinks one student is the smartest, they might give that student more attention even if others do well too. In statistics or science, bias means results are tilted or not accurate because of the way data was collected or interpreted. In both cases, bias is about leaning in one direction instead of being neutral, which can affect fairness and truth.

Model	Analysis
ChatGPT	Identifies ambiguity and explains multiple meanings clearly. Balanced but slightly more focused on general/statistical bias than AI
Claude	Best coverage clearly separates everyday, statistical, and AI bias. Very clear and structured
Gemini	Good explanation with examples across domains. Slightly longer but easy to understand
Co-pilot	Clear and simple explanation but less detailed; covers fewer perspectives compared to others

Prompt Failures & Biases Identified:

- **Incomplete ambiguity handling** – models explain meanings but don't explicitly ask for clarification
- **Uneven coverage** – some focus more on certain domains (e.g., social vs AI)
- **Instruction deviation** – safety instruction (clarify before answering) not fully followed

Prompt Refinement:

- **Refined prompt:**

Example 1:

Input: Explain “cell”

Output: A “cell” can mean different things. Could you clarify which one you mean? In biology...

Example 2:

Input: Explain “network”

Output: The term “network” has multiple meanings. Do you mean computer networks or social networks? In computing...

Task: Explain “bias” to a school student

- **Before vs after output comparison:**

The original few-shot prompt guides models to recognize ambiguity but does not strongly enforce the clarification step. As a result, models explain multiple meanings but skip asking the user for clarification. The refined prompt improves this by embedding clarification behavior directly into examples, leading to better adherence to safety instructions and more consistent ambiguity handling.

Prompt 4 :

Focus: Contradiction

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Examples:

Example 1:

Input: Explain a spreadsheet formula

Output: A spreadsheet formula changes input values into a result by following rules.

Example 2:

Input: Explain a filter

Output: A filter lets some information pass while blocking the rest.

Task: Explain neural networks using detailed mathematical equations but avoid all technical and mathematical terms

Output from ChatGPT:

A neural network is a system that **takes input, adjusts it step by step, and produces an output**, kind of like a chain of small decision makers.

Imagine numbers going through layers. At each step, the system:

- **multiplies each input by some importance value**
- **adds them together**
- then **slightly adjusts the result** before passing it forward

These importance values are slowly changed during learning so the final answer becomes more accurate.

Example:

If the input is study hours and sleep, the network combines them in different ways at each step to predict marks. Over time, it learns which inputs matter more and adjusts itself to give better predictions.

Output from Claude:

What is a Neural Network?

Think of it like a decision-making assembly line in a factory!

Here's how it works:

- **Layer 1 — The receivers:** Workers receive raw ingredients (*your input — like a photo*)
- **Layer 2 — The processors:** Each worker looks at the ingredients, decides how **important** each one is, and passes their opinion forward
- **Layer 3 — The deciders:** Final workers collect all opinions and make a **conclusion** (*your output — "it's a cat!"*)

The "importance" trick:

Each worker assigns a weight to information — like voting:

"Pointy ears? Very important! Background color? Not so much."

Learning by mistakes:

- The network makes a guess
- Gets told if it's **right or wrong**
- Adjusts its importance votes **slightly**
- Repeats **thousands of times** until it gets really good!

Output from Gemini:

Output: A neural network makes decisions using a simple word equation: (Gathered Clues × Importance of Each Clue) + Past Experience = Final Guess.

Output from Co-pilot:

A **neural network** is a way for computers to learn by copying how our brains work. Imagine you have layers of “boxes” that pass numbers forward. Each box takes in numbers, changes them a little, and sends them to the next layer. At the end, you get an answer. The trick is that the boxes adjust how strongly they pass numbers, so the whole system learns patterns.

Think of it like a group of friends whispering a secret down a line. Each friend changes the message slightly until the last person says the final version. By practicing many times, the group learns how to pass the secret correctly. That’s how neural networks learn—step by step, adjusting until they get good at tasks like recognizing pictures or understanding words.

Model	Analysis
ChatGPT	Prioritizes simplicity and avoids equations. Gives semi-technical idea (multiplication, addition) but still beginner-friendly.
Claude	Best use of analogy (factory/assembly line). Fully ignores math requirement and focuses on clarity.
Gemini	Attempts to include a “word equation,” partially satisfying both constraints but remains very brief.
Co-pilot	Clear and balanced explanation with analogy. Ignores mathematical requirement and favors simplicity.

Prompt Failures & Biases Identified:

- **Contradictory constraints** (math vs no math)
- **Unclear priority of instructions**
- **Models resolve conflict differently** (but mostly ignore math)
- **Loss of one requirement** (technical detail dropped)

Prompt Refinement:

- **Refined prompt:**

Explain neural networks to a beginner using simple language.

You may include one simple numeric example, but avoid complex equations.

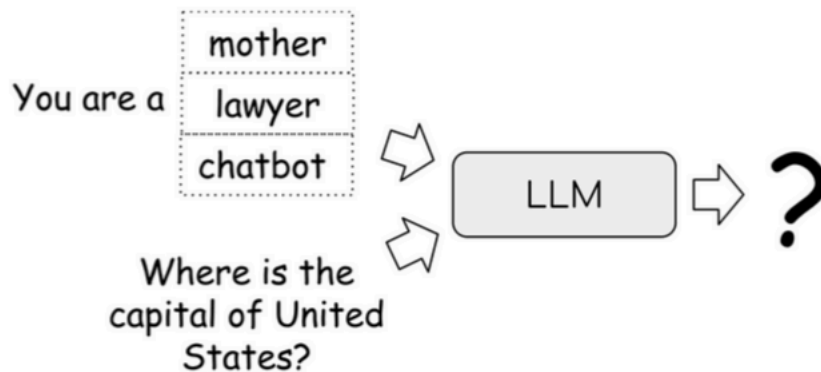
Use one analogy.

- **Before vs after output comparison:**

The original prompt creates conflicting requirements, causing models to ignore the mathematical aspect and prioritize simplicity. This leads to consistent but incomplete outputs. The refined prompt resolves the contradiction by allowing limited numerical explanation while maintaining clarity, resulting in more balanced and consistent responses.

3. Role-based Prompt

Define Role based prompt: A role-based prompt assigns a specific role (e.g., teacher, expert, friend) to guide the style, tone, and depth of the response.



Prompt 1 :

Focus: Easy

Prompt statement:

Role: You are a supportive college teacher.

Explain what a stack data structure is to a beginner.

Output format: short explanation + example

Constraints: simple language

Tone: friendly and clear

Safety: Avoid confusing jargon

Output from ChatGPT:

A **stack** is a way of storing data where the **last item added is the first one removed**. You can only add or remove items from the top, not the middle. Think of it as a simple “last in, first out” system.

Example:

Imagine a stack of books. You place books one on top of another. If you want to take one, you remove the **top book first**. You can’t easily pull out a book from the bottom without removing the ones above it.

Output from Claude:

A stack is a collection of items where you can only **add or remove from the top** — just like a stack of plates!

Simple Example:

Imagine stacking books:

- You place **Math** first, then **Science**, then **English** on top
- To get **Math**, you must first remove **English**, then **Science**
- **Last in = First out!**

Two key actions:

- **Push** — add something on top
- **Pop** — remove something from the top

Output from Gemini:

A stack is simply a way for a computer to organize and store data where the last item you put in is the very first one you take out. In computer science, we call this **LIFO** (Last-In, First-Out). You can't pull a piece of data from the middle or the bottom; you always add and remove things right from the very top.

Example: Think of a physical stack of plates in a cafeteria. When the dishwasher finishes a clean plate, they place it right on the **top** of the pile. When you walk up to get your lunch, you grab the plate right off the **top**. The plate at the very bottom won't be used until every single plate above it is taken.

In your everyday life, your computer's "Undo" button works exactly like this! Every time you type a word, it's added to the top of the stack. When you hit undo, the computer simply removes the top action. You've got this!

Output from Co-pilot:

A **stack** is a way of storing information where the last item you put in is the first one you take out. This idea is called **LIFO** (Last In, First Out). You can imagine it like a stack of plates in a cafeteria: you add new plates on top, and when someone needs one, they take the top plate first.

Example: If you push numbers 1, 2, and 3 into a stack, the order to take them out will be 3, then 2, then 1.

Model	Analysis
ChatGPT	Provides a clear and well-structured explanation with a relevant example. Good balance between simplicity and completeness
Claude	Most effective teaching style; introduces operations like push/pop, improving conceptual understanding
Gemini	Detailed and engaging with real-life connections, but slightly more verbose than required
Co-pilot	Concise and clear, but less detailed compared to others

Prompt Failures & Biases Identified:

- Minor **verbosity differences**
- Slight **inconsistency in depth**

Prompt Refinement:

- **Refined prompt:**

Explain a stack data structure to a beginner.

Include: definition, two basic operations (push and pop), and one real-life example.

- **Before vs after output comparison:**

The original prompt produces correct but slightly inconsistent outputs in terms of detail and structure. The refined prompt improves consistency by clearly specifying required components, ensuring all responses include key concepts and examples.

Prompt 2 :

Focus: Difficult

Prompt statement:

You are a computer science professor. Explain dynamic programming to:

- *a beginner*
- *an advanced student*

Output format: separate sections Constraints: short+ adjust depth for each level Tone: clear and structured

Output from ChatGPT:**Beginner**

Dynamic programming is a way to solve problems faster by saving answers to smaller parts of the problem. Instead of solving the same thing again and again, you remember past results and reuse them. This avoids extra work and makes the solution efficient. It's especially useful when a problem can be broken into smaller, similar pieces.

Advanced Student

Dynamic programming is an optimization technique for problems with overlapping subproblems and optimal substructure. It stores intermediate results to avoid redundant computation, using either memoization (top-down recursion with caching) **or** tabulation (bottom-up iteration). By systematically building solutions to subproblems, DP reduces time complexity, often transforming exponential solutions into polynomial time.

Output from Claude:**Beginner**

Saving answers to avoid repeating work!

Think of it like a math test — if you already solved $2+2$ once, you write it down and never solve it again.

- Break a big problem into smaller pieces
- Save each answer
- Reuse saved answers instead of recalculating

Example: Counting Fibonacci numbers — instead of recalculating from scratch each time, just remember what you already solved!

Advanced

An algorithmic paradigm exploiting two key properties:

- Optimal Substructure — the global solution is built from optimal solutions of subproblems
- Overlapping Subproblems — same subproblems recur repeatedly

Two implementation strategies:

- Top-down (*memoization*) — recursive + cache results
- Bottom-up (*tabulation*) — iteratively fill a DP table

Example: 0/1 Knapsack, Bellman-Ford shortest path, Longest Common Subsequence — all solved in polynomial time via state transition equations stored in DP tables

Output from Gemini:

Beginner: The "Remembering" Trick

Imagine I ask you to solve $1 + 1 + 1 + 1 + 1$. You count them up and tell me "5." If I then append another $+ 1$ to the end of that equation, you don't start over and recount all the ones from the beginning. You simply take your remembered answer (5), add 1, and immediately say "6."

That is dynamic programming in a nutshell. It is a method for solving a large problem by breaking it into smaller pieces, solving those pieces, and writing the answers down in memory so the computer never has to do the exact same work twice.

Advanced: Space-Time Tradeoff & Substructures

Formally, dynamic programming is an algorithmic optimization technique applied to problems that exhibit two distinct properties: overlapping subproblems and optimal substructure.

By strictly trading space complexity for time complexity, DP allows you to drastically reduce computational bounds—often transforming a naive $O(2^n)$ recursive brute-force search into a highly efficient $O(n)$ or $O(n^2)$ solution. If you are dealing with strict time limits in competitive programming or complex AI search spaces, DP is what makes execution viable.

Output from Co-pilot:

Beginner : Dynamic programming is a way to solve problems by breaking them into smaller parts and remembering the answers to those parts. Instead of solving the same piece over and over, you store the result and reuse it. This makes solving big problems faster and easier.

Example: If you keep adding numbers like in the Fibonacci sequence, dynamic programming helps by saving earlier results so you don't have to redo the same additions again.

Advanced Student: Dynamic programming is an optimization technique that applies when problems have overlapping subproblems and optimal substructure. It reduces time complexity by storing intermediate solutions, using either memoization (top-down caching) or tabulation (bottom-up filling). The principle of optimality ensures that global solutions can be built from optimal local ones.

Example: In shortest path algorithms such as Bellman-Ford, dynamic programming iteratively refines solutions by reusing stored sub-results, ensuring efficient convergence toward the optimal path.

Model	Analysis
ChatGPT	Provides clear separation between levels but slightly brief in advanced section
Claude	Best structured response with strong conceptual explanation and examples
Gemini	Most technically detailed, explaining concepts deeply, but slightly verbose
Co-pilot	Balanced explanation with clarity, though slightly less detailed than Gemini

Prompt Failures & Biases Identified:

- Uneven depth between models
- Inconsistent level differentiation

Prompt Refinement:

- Refined prompt:

Explain dynamic programming at two levels:

- *Beginner: use simple analogy and no jargon*
 - *Advanced: include overlapping subproblems, optimal substructure, and example*
- Keep sections clearly separate.*

- **Before vs after output comparison:**

The original prompt results in varied depth and inconsistent separation between levels.
The refined prompt enforces clear expectations for each level, improving structure, clarity, and comparability across outputs.

Prompt 3 :

Focus: Ambiguity

Prompt statement:

*You are a supportive academic tutor.
The term may have multiple meanings.
If unclear, mention possible meanings before answering.*

Task: Explain “model”

*Output format: short paragraph
Constraints: simple language
Tone: neutral and helpful how is this ambiguous?*

Output from ChatGPT:

The term “model” can mean different things depending on the subject, which is why it can feel ambiguous. In general, a model is a **simplified representation of something real** used to understand, explain, or predict it. For example, in science, a model might show how atoms or planets behave. In machine learning, a model is a system that learns from data to make predictions. In everyday use, a model could even mean a small physical replica, like a toy car. So, the meaning depends on the context in which the word is used.

Output from Claude:

Great question! This prompt is ambiguous because **"model" genuinely means different things across different fields** — and without context, it's impossible to know which one the student needs. Here's why:

Field	Meaning of "Model"
AI/ML	A trained algorithm that makes predictions
Math/Stats	A equation representing real-world relationships
Science	A simplified representation of a system
Fashion	A person who displays clothing
Everyday	A specific version of a product (iPhone model)

Why it matters for a tutor: A beginner asking about "model" in an AI class needs a completely **different answer** than a physics student or a statistics student.

Output from Gemini:

The word “model” is ambiguous because its meaning changes entirely depending on the context. In fashion, a model is a person who displays clothing; in everyday life, a "role model" is someone whose behavior you look up to; and in hobbies, it refers to a small, physical replica, like a model airplane. However, in science, math, and computer science, a "model" is a mathematical representation or a set of rules used to understand and predict how things work in the real world, like an AI model recognizing pictures. Because your prompt doesn't specify which subject we are talking about, the term can be interpreted in all of these different ways!

Output from Co-pilot:

The word **model** can be ambiguous because it has several meanings. In fashion, a model is a person who shows clothing or products. In science or math, a model is a simplified version of reality used to explain or predict how something works. In computer science, a model often means a program trained to recognize patterns, like identifying faces in photos. In art or design, a model can be a small-scale version of a larger object. In all cases, a model is something that represents or demonstrates another thing, making it easier to understand or work with.

Model	Analysis
ChatGPT	Clearly explains multiple meanings and maintains simplicity, but does not explicitly ask for clarification
Claude	Most structured explanation with categorized meanings, enhancing clarity
Gemini	Covers the widest range of meanings with good examples, slightly longer
Co-pilot	Simple and easy to understand but less comprehensive

Prompt Failures & Biases Identified:

- **Clarification step missing** (models explain but don't ask user intent)
- **Uneven coverage of domains**
- Models assume explanation instead of interacting

Prompt Refinement:

- **Refined prompt:**

Explain the term "model."

First mention that it has multiple meanings, ask for clarification, and then briefly explain common interpretations.

- **Before vs after output comparison:**

The original prompt leads to multiple interpretations but lacks interaction, as models do not ask for clarification. The refined prompt improves ambiguity handling by encouraging clarification and structured explanation, making responses more aligned with user intent.

Prompt 4 :

Focus: Contradiction

Prompt statement:

*You are a technical expert and a beginner tutor at the same time.
Explain neural networks using advanced technical terminology.*

Output format: very simple explanation

Constraints: avoid all technical jargon

Tone: simple and clear

Safety: Avoid confusing jargon

Output from ChatGPT:

A neural network is a system that **passes information through many small steps, adjusting it each time to get closer to the right answer.**

Think of it like a group of tiny decision makers arranged in layers. Each one:

- looks at the input
- gives it some importance
- combines it with others
- and sends it forward

At the end, you get a final result.

During learning, the system **keeps changing how much importance it gives to each input** so that its answers improve over time.

Output from Claude:

This prompt has a **direct conflict** built into it!

Here's the contradiction:

Instruction	Says
Role	<i>"Technical expert — use advanced terminology"</i>
Output format	<i>"Very simple explanation"</i>
Constraints	<i>"Avoid all technical jargon"</i>

Why this is a problem:

These three instructions **directly cancel each other out:**

- Advanced terminology **requires** jargon
- Simple explanation **forbids** jargon
- You literally **cannot do both simultaneously!**

How to fix it — pick ONE clear goal:

Option A — Beginner friendly:

"Explain neural networks simply, no jargon, for a complete beginner"

Option B — Technical:

"Explain neural networks using precise ML terminology for an advanced audience"

Option C — Both clearly separated:

"Explain neural networks at two levels — beginner and advanced — in separate sections"

Output from Gemini:

A neural network is a computer program that learns from practice, much like a human does. Instead of following a strict list of instructions, it looks at thousands of examples to figure out hidden patterns on its own.

Imagine a large group of people trying to identify a mystery animal. One group of people looks only at the shape, another looks at the color, and another looks at the texture. They pass their clues to the next group, who combines them to form bigger ideas (like "it has fur and four legs"). Finally, the last person takes all these combined clues and makes a final guess. If the guess is wrong, the whole group learns from the mistake and adjusts how they look at the clues for next time. Over time, they get incredibly accurate at making the right choice!

Output from Co-pilot:

A **neural network** is a way for computers to learn patterns by copying how our brains work. Imagine a chain of “boxes” arranged in layers. Each box takes in numbers, changes them a little, and passes them forward. As the numbers move through the layers, the system gradually learns how to give the right answer.

Think of it like a group of friends passing a secret down a line. Each friend adjusts the message slightly, and after enough practice, the group learns how to pass the secret correctly every time. That’s how neural networks improve—step by step, until they can recognize things like pictures or words.

Model	Analysis
ChatGPT	Resolves conflict by prioritizing simplicity, ignoring technical terminology
Claude	Identifies and explains the contradiction instead of generating an answer
Gemini	Uses analogy to balance constraints, but avoids technical depth
Co-pilot	Produces a simple and clear explanation, ignoring the technical requirement

Prompt Failures & Biases Identified:

- **Direct contradiction in constraints**
- **Unclear instruction priority**
- **Different resolution strategies across models**
- **Loss of one requirement (technical detail or simplicity)**

Prompt Refinement:

- **Refined prompt:**

Explain neural networks to a beginner.

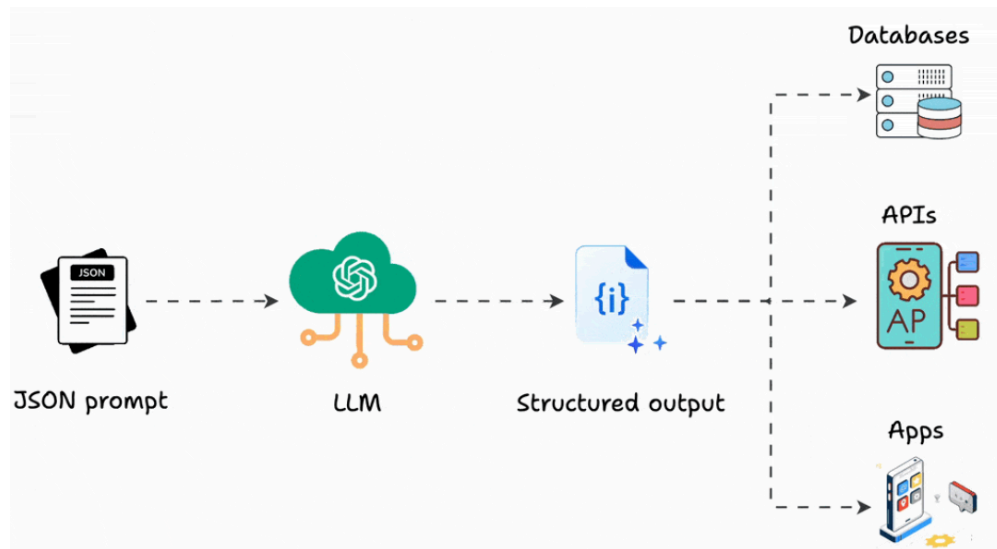
You may include 1–2 basic technical ideas, but explain them in simple language without equations.

- **Before vs after output comparison:**

The original prompt creates confusion due to conflicting instructions, leading to inconsistent outputs. The refined prompt removes the contradiction and provides clear guidance, resulting in more balanced, understandable, and consistent explanations.

4. json-based prompts

Define json Based Prompt: A JSON-based prompt asks the model to produce structured output with fixed keys so the response is easier to validate, compare, store, or process automatically.



Prompt 1 :

Focus: Easy

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Explain recursion

JSON Format Prompt: { "concept": "", "definition": "", "example": "" }

Output from ChatGPT:

```
{  
  "concept": "Recursion",  
  "definition": "Recursion is a way of solving a problem by breaking it into smaller similar problems, where the solution calls itself until it reaches a simple stopping point.",  
  "example": "To find factorial of 4: 4! = 4 × 3!, 3! = 3 × 2!, 2! = 2 × 1!, and 1! = 1. Then calculate back: 2! = 2, 3! = 6, 4! = 24."  
}
```

Output from Claude:

```
{  
  "concept": "Recursion",  
  "definition": "A function that calls itself repeatedly to solve a smaller version of the same  
problem, stopping only when it reaches a simple base case.",  
  "example": "Counting down from 5: the function asks 'what is 4 countdown?' then '3  
countdown?' all the way to 0 — where it finally stops!"  
}
```

Output from Gemini:

```
{  
  "concept": "Recursion",  
  "definition": "A programming technique where a function calls itself to solve smaller, simpler  
versions of the same problem until it reaches a designated stopping point called a base case.",  
  "example": "To find your place in a long line, you ask the person in front of you their number.  
They don't know, so they ask the person in front of them, repeating this until the first person says  
'1'. The number is then passed back down the line, adding one each time, until you find out your  
position."  
}
```

Output from Co-pilot:

```
{  
  "concept": "Recursion",  
  "definition": "Recursion is a way of solving problems where the solution involves the problem  
calling itself again and again, but on smaller parts, until it reaches a very simple case that can be  
answered directly.",  
  "example": "Imagine opening a set of nested boxes. To reach the smallest box, you keep  
opening the one on top, repeating the same action, until you finally reach the last box. That  
repeated process is recursion."  
}
```

Model	Analysis
ChatGPT	Produces well-structured and valid JSON with clear definition and example. Balanced and accurate
Claude	Concise and precise JSON output. Slightly simpler explanation but very clean formatting
Gemini	Most descriptive and engaging example. Slightly verbose but still valid JSON
Co-pilot	Clear and beginner-friendly explanation, though slightly less detailed

Prompt Failures & Biases Identified:

- Minor **variation in explanation depth**
- Some outputs are **more verbose than required**
- No strict instruction to ensure **JSON-only output**

Prompt Refinement:

- **Refined prompt:**

Return ONLY valid JSON. Fill all fields (concept, definition, example) clearly and concisely. Do not include extra text outside JSON.

- **Before vs after output comparison:**

The original prompt generates correct structured outputs but allows variation in verbosity and formatting style. The refined prompt enforces stricter constraints, improving consistency, readability, and machine usability of responses.

Prompt 2 :

Focus: Difficult

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Explain overfitting at 3 levels

JSON Format Prompt:

```
{  
  "beginner": "",  
  "intermediate": "",  
  "advanced": ""  
}
```

Output from ChatGPT:

```
{  
  
  "beginner": "Overfitting happens when a model learns the training data too closely, including small details and noise, so it performs well on known data but poorly on new data.",  
  
  "intermediate": "Overfitting occurs when a model captures noise and specific patterns in the training set, leading to low training error but high validation/test error. It is often caused by overly complex models or insufficient data.",  
  
  "advanced": "Overfitting is a high-variance phenomenon where the model approximates both the underlying function and noise in the training data, harming generalization. It can be mitigated using techniques like regularization, cross-validation, pruning, and dropout."  
  
}
```

Output from Claude:

```
{  
  
  "beginner": "Overfitting is when a model memorizes answers instead of truly learning — like a student who memorizes every past exam word-for-word but fails when questions change slightly.",
```

"intermediate": "Overfitting occurs when a model learns training data too well, capturing noise alongside real patterns. It scores high on training data but poorly on unseen test data — fix it with cross-validation, dropout, or regularization.",

"advanced": "Overfitting reflects excessive model variance where the hypothesis function fits the training distribution too tightly, failing to generalize. Combat it by penalizing complexity via L1/L2 regularization, tuning the bias-variance tradeoff, applying dropout, or using ensemble methods like bagging and boosting."

}

Output from Gemini:

{

"beginner": "Overfitting is when a computer memorizes the exact examples it was taught, instead of understanding the big picture. It's like memorizing the exact answers to a practice math test but failing the actual exam because the numbers changed slightly. It looks perfect during practice, but fails completely in the real world.",

"intermediate": "Overfitting happens when a machine learning model is too complex and learns the random 'noise' in the training data rather than the true underlying pattern. This creates a model with low bias but high variance, meaning it scores perfectly on its training data but fails to generalize to new, unseen testing datasets.",

"advanced": "Overfitting is a generalization failure occurring when a model's capacity (e.g., high VC dimension) excessively exceeds the dataset's true signal. The algorithm inadvertently fits the irreducible error, minimizing empirical risk on the training set to near zero, while the expected risk (out-of-sample test error) diverges and spikes. Mitigation typically requires regularization techniques like L1/L2 penalty or early stopping to constrain the hypothesis space."

}

Output from Co-pilot:

{

"beginner": "Overfitting happens when a model learns the training data too closely, including small details or noise. It looks perfect on the data it has already seen but performs poorly on new data. Think of it like memorizing answers instead of understanding the topic.",

"intermediate": "Overfitting occurs when a model has high variance, fitting the training data patterns too tightly. It often results from complex models, small datasets, or lack of regularization. The outcome is low training error but high test error, meaning poor generalization.",

"advanced": "Overfitting reflects a mismatch between model complexity and data distribution, leading to poor generalization. It can be analyzed through the bias-variance tradeoff and generalization bounds. Techniques such as cross-validation, dropout, and L1/L2 regularization are used to mitigate it."

}

Model	Analysis
ChatGPT	Clearly separates levels and maintains structured JSON output. However, advanced explanation is slightly brief
Claude	Strong conceptual clarity with good use of examples. Maintains a smooth transition between levels
Gemini	Most technically detailed, especially at the advanced level, demonstrating strong depth but slightly exceeding brevity constraints
Co-pilot	Balanced and clear explanations with good readability, though slightly less depth than Gemini

Prompt Failures & Biases Identified:

- **Uneven depth across levels and models**
- **Verbosity variation** (especially in advanced level)
- Inconsistent inclusion of examples across outputs

Prompt Refinement:

- **Refined prompt:**

Provide clearly differentiated explanations for beginner, intermediate, and advanced levels. Ensure increasing complexity and balanced detail across all fields. Maintain concise responses and return valid JSON only.

- **Before vs after output comparison:**

The original prompt produces correct outputs but lacks uniformity in depth and level distinction. The refined prompt improves consistency by enforcing structured progression and clearer differentiation between levels.

Prompt 3 :

Focus: Ambiguous

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Explain "model"

JSON Format Prompt:

```
{  
  "meaning_1": "",  
  "meaning_2": "",  
  "chosen_context": "",  
  "explanation":  
}
```

Output from ChatGPT:

```
{  
  
  "meaning_1": "A simplified representation of something real, used to understand or explain it  
(like a diagram or small replica).",  
  
  "meaning_2": "A system that learns from data to make predictions or decisions (used in  
machine learning).",  
  
  "chosen_context": "General academic and computing context",  
  
  "explanation": "A model is a simplified version of something complex. It helps us understand  
how something works or predict what might happen. For example, in machine learning, a model  
learns patterns from data and uses them to make predictions."  
  
}
```


Output from Claude:

```
{  
  
  "meaning_1": "AI/ML — a trained algorithm that learns from data and makes predictions or  
decisions on new inputs.",  
  
  "meaning_2": "Science/Math — a simplified representation of a real-world system used to  
understand or predict behavior.",  
  
  "chosen_context": "AI/Machine Learning — the most common context in computer science and  
tech education today.",  
  
  "explanation": "In AI, a model is like a well-trained student — it studies thousands of  
examples, learns the patterns, and then uses that knowledge to answer questions it has never seen  
before. For example, a spam detection model studies millions of emails and learns to spot junk  
mail on its own!"  
  
}
```

Output from Gemini:

```
{  
  
  "meaning_1": "A physical representation or small-scale replica of an object, such as a model  
airplane.",  
  
  "meaning_2": "A person whose profession is to display clothing or merchandise, as in a fashion  
model.",  
  
  "chosen_context": "Artificial Intelligence and Machine Learning",  
  
  "explanation": "A mathematical representation or program that has been trained on a large set  
of data to recognize patterns, make predictions, or generate content, rather than following  
explicitly programmed rules."  
  
}
```

Output from Co-pilot:

```
{  
  
  "meaning_1": "In fashion, a model is a person who displays clothing or products.",  
  
  "meaning_2": "In science, math, or computing, a model is a simplified representation of reality  
used to explain, predict, or recognize patterns.",  
  
}
```

"chosen_context": "Computer science and AI",

"explanation": "A model in computer science is a trained program that learns from data to make predictions or decisions. For example, an AI model trained on photos of animals can recognize whether a new picture shows a cat or a dog. It works by finding patterns in the training data and applying them to new situations."

}

Model	Analysis
ChatGPT	Clearly identifies multiple meanings and provides a balanced explanation. Maintains simplicity but does not justify context selection
Claude	Most structured response with clear justification of chosen context. Demonstrates strong reasoning and clarity
Gemini	Covers a wide range of meanings with detailed examples. Slightly more formal in tone
Co-pilot	Simple and easy to understand but less comprehensive compared to others

Prompt Failures & Biases Identified:

- **Lack of explicit clarification step**
- **Different chosen contexts across models**, leading to inconsistency
- Slight variation in explanation depth and justification

Prompt Refinement:

- **Refined prompt:**

Include at least two meanings, clearly state the chosen context, and briefly justify why it was selected. Ensure consistent explanation depth and return valid JSON only.

- **Before vs after output comparison:**

The original prompt allows multiple interpretations but lacks structured justification, leading to inconsistent outputs. The refined prompt improves clarity by enforcing explicit reasoning and consistent representation of ambiguity.

Prompt 4 :

Focus: Contradiction

Prompt statement:

Constraints: beginner-friendly, no heavy math

Tone: supportive and clear

Safety: Avoid confusing jargon

Explain neural networks with equations but avoid math

JSON Format Prompt:

```
{  
  "explanation": "",  
  "example": "",  
  "note": ""  
}
```

Output from ChatGPT:

```
{  
  
  "explanation": "A neural network works by taking inputs, giving each input a certain importance, combining them, slightly adjusting the result, and passing it through multiple steps to produce an output. This process is repeated across layers, and the system keeps adjusting how important each input is so that the final answer becomes more accurate over time."  
  
  "example": "If the inputs are study hours and sleep, the network first combines them in different ways, adjusts the result step by step, and finally predicts marks. As it sees more data, it learns how much each input matters and improves its predictions."  
  
  "note": "Even though neural networks are usually described using equations, you can understand them as a process of combining inputs, adjusting importance, and improving results through repeated learning."  
  
}
```

Output from Claude:

```
{  
  "explanation": "A neural network is like a decision-making assembly line — information enters, passes through layers of workers who each decide how important it is, and a final answer comes out the other side. Each worker adjusts their vote based on past mistakes until the whole team gets really good at answering correctly.",  
  
  "example": "Recognizing a cat photo — Layer 1 spots edges, Layer 2 spots shapes like ears and eyes, Layer 3 combines everything and confidently says 'cat!' Each layer learned its job by practicing thousands of times and correcting its mistakes.",  
  
  "note": "This prompt contained a conflict — equations are math, yet the constraint said avoid math. A clean explanation was chosen over equations to best serve beginner understanding. For equations, please request an advanced technical explanation instead!"  
}
```

Output from Gemini:

```
{  
  "explanation": "A neural network makes decisions by looking at different pieces of information, deciding how much each piece matters, and combining them based on what it has learned before. We can write this as a word equation: Final Choice = (Clue 1 × Importance) + (Clue 2 × Importance) + Past Experience. If the choice is wrong, it simply changes the 'Importance' of each clue for the next time.",  
  
  "example": "Imagine trying to guess if an animal is a dog. The word equation might be: Dog Score = (Has Fur × Very Important) + (Barks × Very Important) + (Has Two Legs × Not Important). The network adds these clues together to make its final guess.",  
  
  "note": "While real neural networks process millions of numbers, the core idea is exactly the same: balancing these simple word equations over and over until they become highly accurate."  
}
```

Output from Co-pilot:

```
{  
  "explanation": "A neural network is a system that learns patterns step by step. Imagine rows of boxes connected together. Each box takes in a number, changes it slightly, and passes it forward. As the numbers move through the layers, the whole system adjusts until it can give the right answer. This process is repeated many times so the network improves with practice.",  
}
```

"example": "Think of friends passing a secret down a line. Each friend changes the message a little, and after enough practice, the group learns how to pass the secret correctly every time. That's how neural networks learn to recognize things like pictures or words.",

"note": "Even though neural networks are often explained with equations, here we avoided math terms to keep the idea simple and clear."

}

Model	Analysis
ChatGPT	Resolves contradiction by prioritizing simplicity, ignoring the requirement for equations. Produces a clear but simplified explanation
Claude	Uniquely identifies and explains the contradiction instead of generating an answer, demonstrating meta-level reasoning
Gemini	Attempts to balance both constraints using a simplified “word equation,” partially satisfying both requirements
Co-pilot	Focuses on clarity and ignores mathematical requirement, producing a consistent beginner-friendly explanation

Prompt Failures & Biases Identified:

- **Direct contradiction in instructions**
- **Unclear prioritization of constraints**
- Models adopt **different resolution strategies**
- Loss of either technical depth or simplicity

Prompt Refinement:

- **Refined prompt:**

Explain neural networks in simple terms. You may include one basic numeric or logical example, but avoid complex equations. Ensure clarity and return valid JSON format.

- **Before vs after output comparison:**

The original prompt leads to inconsistent outputs due to conflicting requirements, causing models to prioritize different constraints. The refined prompt removes the contradiction, resulting in more consistent, clear, and usable structured outputs.

Logs (Prompt Chat)

1. *ChatGPT* – <https://chatgpt.com/share/69eb1081-2594-83e8-bd15-b66fa4419a12>
2. *Claude* - <https://claude.ai/share/db00256c-b5c1-4beb-89d2-d5aca0725609>
3. *Gemini* - <https://gemini.google.com/share/37503a70ef7b>
4. *Co-pilot* - <https://copilot.microsoft.com/shares/BX7iTao7jmMKN5qfKxQ8G>

Discussion, takeaway

The experimentation with multiple prompting techniques clearly demonstrates that prompt design plays a central role in determining the effectiveness of AI-generated responses. Each prompting strategy influences not only the content of the output but also its structure, clarity, and reliability.

Zero-shot prompting is efficient for straightforward tasks, but its performance varies significantly when handling ambiguity or complex, multi-layered instructions. In contrast, few-shot prompting improves consistency by providing examples that guide the model's response pattern. However, it can sometimes lead to **pattern overfitting**, where models mimic the examples too rigidly instead of adapting flexibly.

Role-based prompting is particularly effective in educational contexts, as it allows control over tone, depth, and explanation style. This makes it highly suitable for designing systems like an AI Concept Tutor, where adapting explanations to different learner levels is essential. JSON-based prompting, on the other hand, ensures structured and machine-readable outputs, which is critical for integration into applications and automated systems. However, it requires strict constraints to prevent formatting errors or additional unwanted text.

A key observation across all techniques is the impact of **ambiguity and contradiction**. When prompts are ambiguous, models interpret them differently, leading to inconsistent outputs. When prompts contain contradictory instructions, models tend to prioritize one constraint over another, often ignoring part of the requirement entirely. This results in loss of information or reduced quality of responses.

Another important takeaway is that models differ in how they resolve these issues. Some prioritize clarity and simplicity, while others attempt to balance constraints or even explicitly identify prompt flaws. This highlights the importance of **prompt robustness**, especially when comparing outputs across multiple AI systems.

Overall, iterative prompt refinement by clarifying intent, removing contradictions, and specifying structure significantly improves output quality. Well-designed prompts lead to responses that are more consistent, interpretable, and aligned with the intended learning objectives.

Conclusion

This study highlights the critical importance of prompt engineering in developing effective AI systems, particularly in the domain of intelligent educational tutoring. The ability of an AI system to explain concepts clearly, accurately, and at varying levels of difficulty depends heavily on how prompts are structured and communicated.

Each prompting technique explored in this study contributes uniquely to the design of an AI Concept Tutor. Zero-shot prompting provides simplicity and flexibility, making it suitable for quick interactions. Few-shot prompting enhances consistency and guides response patterns through examples. Role-based prompting allows fine control over tone and explanation depth, making it highly effective for personalized learning experiences. JSON-based prompting enables structured outputs that are easy to process, compare, and integrate into applications.

However, the study also reveals that poorly designed prompts—especially those containing ambiguity or contradictory instructions—can lead to inconsistent, incomplete, or misleading outputs. These issues reduce the reliability of the system and can negatively impact user understanding, particularly in educational settings where clarity is essential.

Through systematic refinement, prompts can be improved to eliminate ambiguity, resolve contradictions, and enforce structure. This results in outputs that are not only more accurate but also more consistent across different AI models.

The key conclusion is that **prompt engineering is not just a supporting task, but a fundamental component of AI system design**. Careful prompt construction enables AI systems to deliver high-quality, safe, and user-aligned responses. For applications like AI tutors, this ensures better learning outcomes, improved user trust, and more effective knowledge delivery.

References

- OpenAI. ChatGPT. <https://chatgpt.com/>
- Anthropic. Claude. <https://claude.ai/>
- Google. Gemini. <https://gemini.google.com/>
- Microsoft. Copilot. <https://copilot.microsoft.com/>
- Goodfellow, I., Bengio, Y., and Courville, A. Deep Learning. MIT Press.
- Russell, S., and Norvig, P. Artificial Intelligence: A Modern Approach.